Relazione progetto libmr

Gabriele Grazzini 693100

A.A. 2025–2026

Indice

1 Descrizione dell'architettura generale del framework **2**

2 Descrizione dell'interfaccia pubblica **2**

3 Organizzazione dei processi e uso di fork(), pipe(), dup2() e waitpid() **3**

4 Organizzazione dei thread C11 nel mapper e nel reducer **3**

5 Struttura delle code interne e meccanismi di sincronizzazione **4**

6 Macro di controllo degli errori **4**

7 Formato dei messaggi scambiati sulle pipe **5**
7.1 Messaggio main -> mapper 5
7.2 Messaggio mapper -> reducer 5
7.3 Messaggio reducer -> main 6

8 Struttura dati usata per il raggruppamento per token **6**

9 Formato del file di output **8**

10 Formato del file di log **8**

11 Descrizione dei test realizzati **9**
11.1 Test word_count 9
11.2 Test word_length 9

1 Descrizione dell'architettura generale del framework

Il progetto realizza una libreria statica `libmr.a`, usabile da programmi C tramite l'header pubblico `mr.h`.

Il framework esegue una computazione MapReduce locale su file di testo. Il programma utente fornisce una funzione mapper e una funzione reducer. Il framework si occupa di leggere l'input, creare i processi, gestire le pipe, avviare i thread C11, raggruppare i valori per token e produrre il file di output.

L'architettura è composta da tre processi:

- processo principale: legge i file di input e scrive il file di output;
- processo mapper: riceve righe, invoca i mapper worker e produce coppie intermedie;
- processo reducer: riceve coppie, raggruppa per token, invoca i reducer worker e produce risultati finali.

La comunicazione tra processi avviene solo tramite pipe. La fine dei dati viene indicata chiudendo le pipe, non usando messaggi speciali.

2 Descrizione dell'interfaccia pubblica

L'interfaccia pubblica è definita in `include/mr.h`. Il tipo principale è `mr_t`, handle opaco che rappresenta un'elaborazione.

La configurazione usa `mr_attr_t`, che contiene:

- `mapper_threads`: numero di thread mapper;
- `reducer_threads`: numero di thread reducer;
- `queue_size`: capacità delle code interne;
- `log_file`: nome del file di log.

I valori di default sono:

- `mapper_threads = 4`;
- `reducer_threads = 4`;
- `queue_size = 64`;
- file di log `mr.log`.

La riga passata al mapper è rappresentata da `mr_file_line_t`. I campi `file_name` e `line` non devono essere considerati stringhe terminate da `\0`; la loro lunghezza è indicata dai campi `file_name_len` e `line_len`.

Il mapper emette coppie tramite `mr_emit_pair_t`. Il reducer emette risultati finali tramite `mr_emit_result_t`. I valori intermedi e i risultati sono trattati come byte opachi.

Le funzioni principali sono:

- `mr_attr_init()`;
- `mr_attr_destroy()`;
- `mr_create()`;
- `mr_start()`;
- `mr_destroy()`.

3 Organizzazione dei processi e uso di fork(), pipe(), dup2() e waitpid()

All'avvio di `mr_start()` vengono create tre pipe:

- `main_to_mapper`: dal processo principale al mapper;
- `mapper_to_reducer`: dal mapper al reducer;
- `reducer_to_main`: dal reducer al processo principale.

Il processo mapper viene creato con `fork()`. Nel figlio mapper, `dup2()` collega:

- `main_to_mapper[0]` a `STDIN_FILENO`;
- `mapper_to_reducer[1]` a `STDOUT_FILENO`.

Il processo reducer viene creato con un secondo `fork()`. Nel figlio reducer, `dup2()` collega:

- `mapper_to_reducer[0]` a `STDIN_FILENO`;
- `reducer_to_main[1]` a `STDOUT_FILENO`.

Dopo le duplicazioni vengono chiusi tutti i descrittori non necessari. Questo è necessario per permettere la corretta ricezione dell'EOF sulle pipe.

Il processo principale:

- scrive le righe serializzate sulla pipe verso il mapper;
- chiude la pipe quando l'input è finito;
- legge i risultati dal reducer;
- scrive il file di output;
- attende i figli con `waitpid()`.

4 Organizzazione dei thread C11 nel mapper e nel reducer

Il processo mapper usa thread C11 tramite `threads.h`. La sua organizzazione è:

- un thread reader legge le righe da `stdin`;
- una coda interna contiene le righe lette;
- più worker mapper estraggono righe dalla coda;
- ogni worker invoca la funzione mapper utente;
- le coppie prodotte vengono scritte su `stdout`.

Le scritture dei worker mapper verso la pipe sono protette da un mutex, così i messaggi non si mescolano.

Il processo reducer non usa un thread reader separato. Il processo reducer legge direttamente da `stdin` tutte le coppie prodotte dal mapper e costruisce i gruppi per token prima di avviare i worker reducer. Questa scelta evita un thread aggiuntivo non necessario, perché la fase di lettura e raggruppamento deve comunque completarsi prima della riduzione.

Dopo il raggruppamento, il reducer usa:

- una coda interna che contiene i gruppi già completi;
- più worker reducer che estraggono gruppi dalla coda;
- ogni worker invoca la funzione reducer utente;
- i risultati vengono memorizzati nel gruppo.

Il thread principale del processo reducer inserisce i gruppi nella coda e la chiude quando tutti i gruppi sono stati inseriti. Alla fine, dopo la terminazione dei worker, i risultati sono scritti in ordine lessicografico per token su `stdout`.

5 Struttura delle code interne e meccanismi di sincronizzazione

Le code interne sono code circolari a capacità fissa. La capacità è il valore `queue_size` configurato dall'utente.

Ogni coda contiene:

- un array di puntatori `void **buf`;
- indici `head` e `tail`;
- numero di elementi presenti;
- flag `closed`;
- mutex C11 `mtx_t`;
- condition variable C11 `cnd_t not_full`;
- condition variable C11 `cnd_t not_empty`.

Se la coda è piena, il produttore attende su `not_full`. Se la coda è vuota, il consumatore attende su `not_empty`. Quando la coda viene chiusa, tutti i thread in attesa vengono risvegliati.

6 Macro di controllo degli errori

Per rendere uniforme la gestione degli errori ho definito alcune macro in `macros.h`. Le macro eseguono un controllo, scrivono un messaggio nel log in caso di errore e saltano alla sezione di cleanup della funzione tramite `goto fail`. In questo modo il codice evita ripetizioni e libera le risorse in un unico punto.

Le macro usate sono:

- `CHECK_MINUS_ONE(call, message)`: esegue `call` e controlla se il risultato è `-1`. Viene usata per chiamate di sistema come `pipe()`, `dup2()`, `waitpid()`, `read()` e `write()`.
- `CHECK_NULL(ptr, message)`: controlla se un puntatore è `NULL`. Viene usata dopo allocazioni dinamiche, aperture di file o inizializzazioni che restituiscono puntatori.
- `CHECK_READ_FULL_OR_EOF(call, expected, message)`: controlla una lettura che può terminare con EOF. Se il risultato è `0`, salta a `eof`; se il numero di byte letti è diverso da `expected`, segnala errore.
- `CHECK_READ_WRITE_FULL(call, expected, message)`: controlla che una lettura o scrittura abbia trasferito esattamente il numero di byte atteso. Se il risultato è diverso da `expected`, segnala errore.

- `CHECK_THRD_SUCCESS(call, message)`: controlla che una funzione C11 sui thread restituisca `thrd_success`. Viene usata per chiamate come `thrd_create()` e `thrd_join()`.

Le macro non liberano direttamente memoria o descrittori. Il rilascio delle risorse avviene nei blocchi `fail` ed eventualmente `eof` delle singole funzioni. Per questo motivo le funzioni che usano queste macro devono avere le etichette necessarie.

7 Formato dei messaggi scambiati sulle pipe

Tutti i messaggi usano lunghezze esplicite. Le letture e scritture parziali sono gestite con funzioni ausiliarie equivalenti a `readn()` e `writen()`.

7.1 Messaggio main -> mapper

Il messaggio usa un header esplicito di tipo `mr_line_header_t`:

```
typedef struct {
    int file_name_len;
    unsigned long line_number;
    int line_len;
} mr_line_header_t;
```

Il record completo è:

```
mr_line_header_t header
file_name_len byte di file_name
line_len byte di line
```

`file_name_len` deve essere maggiore di zero. `line_len` può essere zero. Il carattere `\n` non viene inviato.

7.2 Messaggio mapper -> reducer

Il messaggio usa un header esplicito di tipo `mr_pair_header_t`:

```
typedef struct {
    int token_len;
    int value_len;
} mr_pair_header_t;
```

Il record completo è:

```
mr_pair_header_t header
token_len byte di token
value_len byte di valore opaco
```

`token_len` non include il terminatore `\0`. Il reducer aggiunge localmente il terminatore per usare il token come stringa C.

I limiti applicati sono:

- $0 <$ `token_len` ≤ 4096;
- $0 \leq$ `value_len` $\leq 64 \cdot 1024 \cdot 1024$;

7.3 Messaggio reducer -> main

Il messaggio usa un header esplicito di tipo `mr_result_header_t`:

```
typedef struct {
    int token_len;
    int result_len;
} mr_result_header_t;
```

Il record completo è:

```
mr_result_header_t header
token_len byte di token
result_len byte di risultato opaco
```

`token_len` non include il terminatore `\0`. `result_len` può essere zero. Il risultato non viene interpretato dal framework.

Anche questo header viene controllato dal processo principale prima di usare le lunghezze ricevute. I limiti applicati sono:

- $0 <$ `token_len` ≤ 4096;
- $0 \leq$ `result_len` $\leq 64 \cdot 1024 \cdot 1024$;

8 Struttura dati usata per il raggruppamento per token

Il reducer raggruppa le coppie usando una tabella hash. La tabella è composta da un array di bucket; ogni bucket contiene una lista collegata di gruppi. Ogni gruppo è rappresentato dal tipo `mr_group_t`. I valori intermedi e i risultati prodotti dal reducer sono memorizzati in liste di nodi `mr_byte_node_t`.

La tabella hash è rappresentata dal tipo `mr_group_table_t`:

```
typedef struct {
    mr_group_t **buckets;
    size_t buckets_count;
    size_t groups_count;
} mr_group_table_t;
```

Il campo `buckets` è un array di puntatori a `mr_group_t`. Ogni posizione dell'array rappresenta un bucket. Se più token producono lo stesso indice hash, i gruppi corrispondenti vengono collegati tramite il campo `next`.

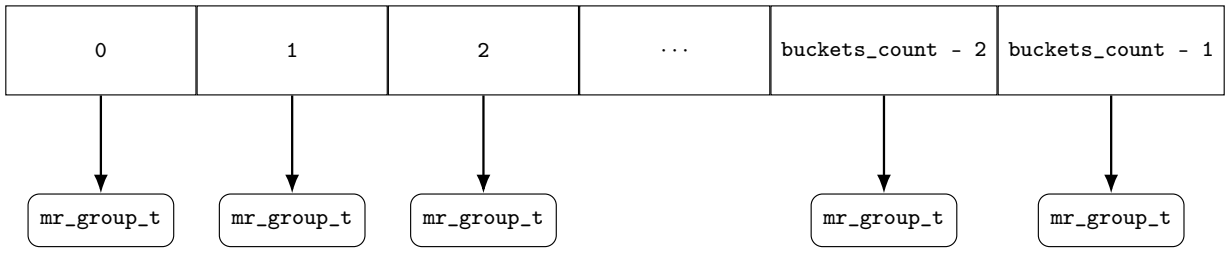


Figura 1: Array dei bucket della tabella hash. Ogni elemento dell'array contiene un puntatore al primo gruppo della lista di collisione del bucket.

L'inserimento di una coppia `<token, valore>` calcola prima l'indice del bucket tramite una funzione hash deterministica. Se nel bucket esiste già un gruppo con lo stesso token, il valore viene aggiunto alla lista `values`. Se il token non è presente, viene creato un nuovo gruppo e inserito nella lista del bucket.

Funzione di hash utilizzata

Per scegliere il bucket in cui inserire un token viene usata la funzione interna `hash_token()`. La funzione scorre i byte del token e costruisce un valore numerico deterministico con una formula polinomiale:

```
hash = hash * 31 + (unsigned char)token[i];
```

Il valore iniziale di `hash` è zero. Per ogni carattere del token, il valore precedente viene moltiplicato per 31 e poi viene sommato il byte corrente. La moltiplicazione per 31 fa sì che l'ordine dei caratteri influenzi il risultato: per esempio token composti dagli stessi caratteri ma in ordine diverso producono, in generale, valori hash diversi.

L'indice effettivo del bucket viene poi calcolato applicando il modulo rispetto al numero di bucket della tabella:

```
bucket = hash_token(token, token_len) % table->buckets_count;
```

Schema del nodo usato per valori e risultati:

```
typedef struct mr_byte_node {
    void *data;
    size_t len;
    struct mr_byte_node *next;
} mr_byte_node_t;
```

Schema del gruppo associato a un token:

```
typedef struct mr_group {
    char *token;
    size_t token_len;

    mr_byte_node_t *values_head;
    mr_byte_node_t *values_tail;
    size_t values_count;

    mr_byte_node_t *results_head;
    mr_byte_node_t *results_tail;
    size_t results_count;

    struct mr_group *next;
} mr_group_t;
```

Ogni `mr_group_t` contiene un token distinto. Il campo `next` collega gruppi appartenenti allo stesso bucket, quindi la tabella hash non mantiene un ordine lessicografico globale. Dopo la lettura di tutte le coppie, il reducer crea un array di puntatori a `mr_group_t` e lo ordina con `qsort()` usando il token. Questo array ordinato viene usato per garantire l'output deterministico.

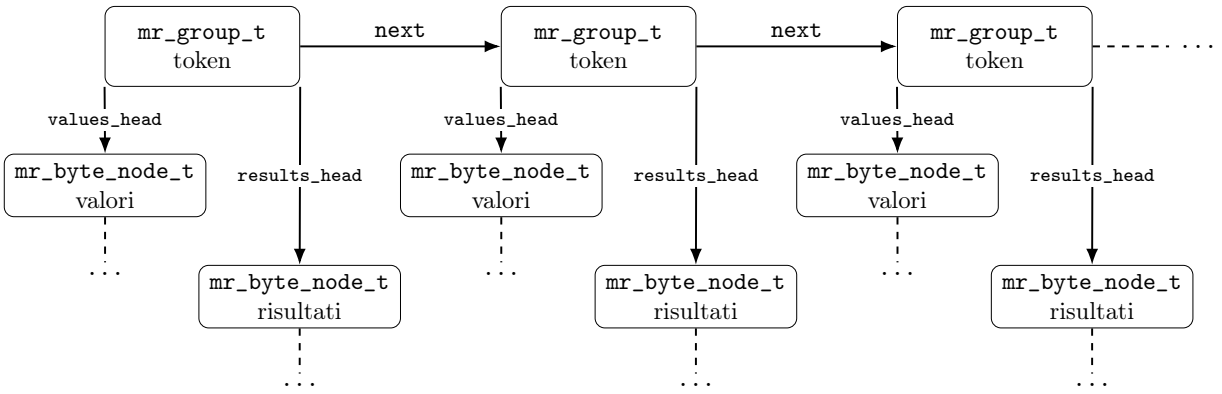


Figura 2: Lista di collisione di un bucket della tabella hash. Ogni `mr_group_t` rappresenta un token e punta separatamente alla lista dei valori intermedi e alla lista dei risultati.

Ogni valore intermedio viene copiato in un nodo della lista `values`. Ogni risultato prodotto dal reducer viene copiato in un nodo della lista `results`. Il framework non interpreta il contenuto di questi byte: usa solo `len` per sapere quanti byte leggere o scrivere.

9 Formato del file di output

Il file di output è binario. Ogni record contiene:

```
int token_len
token_len byte di token
int result_len
result_len byte di risultato
```

Il token non include il terminatore `\0`. Il risultato è una sequenza opaca di byte e può contenere byte nulli.

I record sono scritti in ordine lessicografico per token. Se un reducer emette più risultati per lo stesso token, questi vengono scritti nell'ordine in cui sono stati emessi dal reducer.

10 Formato del file di log

Il framework produce un file di log. Se l'utente non specifica un nome, viene usato `mr.log`.

Il formato di ogni riga è:

```
[tempo] [pid] [thread] [evento] messaggio
```

Il campo `tempo` indica il tempo relativo dall'inizio dell'elaborazione. Il campo `pid` identifica il processo. Il campo `thread` identifica il thread C11.

Il log registra eventi come:

- creazione delle pipe;
- creazione dei processi mapper e reducer;
- avvio e terminazione dei thread;
- apertura e chiusura dei file;
- numero di righe inviate;
- numero di coppie prodotte;
- numero di token distinti raggruppati;

- numero di risultati finali prodotti;
- errori.

Le scritture sul log sono sincronizzate con un semaforo POSIX nominato `/mr_log`.

11 Descrizione dei test realizzati

11.1 Test word_count

Il test `word_count` controlla il caso classico del conteggio delle parole. Il programma usa il framework come libreria esterna: inizializza gli attributi, crea un oggetto `mr_t`, avvia l'elaborazione con `mr_start()` e poi legge il file di output binario per stamparlo in forma leggibile.

Il mapper scorre ogni riga usando `line_len`, quindi non assume che la riga sia terminata da `\0`. Ogni sequenza alfanumerica viene considerata un token, convertita in minuscolo e inviata con valore intero `1`. Il reducer riceve tutti i valori associati allo stesso token, li interpreta come interi e li somma.

Per esempio, con una riga come:

```
Ciao ciao prova
```

il mapper emette:

```
<ciao, 1>
<ciao, 1>
<prova, 1>
```

Il reducer riceve due valori per `ciao` e uno per `prova`, quindi produce:

```
ciao 2
prova 1
```

Questo test mostra che il framework legge l'input, trasporta coppie intermedie, raggruppa correttamente per token e produce un output ordinato.

11.2 Test word_length

Il test `word_length` usa una logica diversa dal conteggio delle singole parole. Anche qui il mapper estrae token alfanumerici dalla riga, ma non usa il token trovato come chiave. Al suo posto crea una chiave della forma `lenN`, dove `N` è la lunghezza del token. Il valore emesso è sempre l'intero `1`.

Il reducer somma tutti gli interi ricevuti per ogni chiave `lenN`. Il risultato indica quante parole hanno quella lunghezza.

Per esempio, con l'input:

```
fork pipe dup2 in C
```

il mapper calcola le lunghezze:

```
fork -> len4
pipe -> len4
dup2 -> len4
in   -> len2
C    -> len1
```

e quindi emette:

```
<len4, 1>
<len4, 1>
<len4, 1>
<len2, 1>
<len1, 1>
```

Il reducer raggruppa le coppie con la stessa chiave e produce:

```
len1 1
len2 1
len4 3
```

Con l'input completo del test, parole diverse con la stessa lunghezza finiscono nello stesso gruppo. Per esempio `fork`, `pipe` e `dup2` hanno tutte lunghezza 4, quindi contribuiscono allo stesso token `len4`.